

Hola, acabo de crear una máquina virtual Alpine Virtual Box, usuarios root y osvaldo, al hacer ssh aparece:

```
PS C:\Users\locant> ssh osvaldo@192.168.1.39
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
```

El problema ocurre porque la IP `192.168.1.39` estuvo asociada anteriormente a otra máquina o instalación, por lo que tu cliente de SSH en Windows detecta una huella (*fingerprint*) distinta en `known_hosts` para evitar ataques de tipo *Man-in-the-Middle*.

Para solucionarlo y volver a conectarte, debes eliminar la entrada antigua guardada en tu archivo `known_hosts`.

Opción 1: Mediante el comando `ssh-keygen` (Recomendado)

Ejecuta el siguiente comando desde PowerShell o la consola de Windows:

PowerShell



```
ssh-keygen -R 192.168.1.39
```

Este comando borrará automáticamente la entrada correspondiente a esa IP en `C:\Users\locant\.ssh\known_hosts`.

Opción 2: Edición manual del archivo

1. Abre el archivo ubicado en `C:\Users\locant\.ssh\known_hosts` con cualquier editor de texto (como Notepad o VS Code).
2. Ve a la línea 26 (como indica el propio error: `Offending ECDSA key in C:\Users\locant\.ssh\known_hosts:26`).
3. Borra esa línea completa, guarda los cambios y cierra el editor.

Una vez ejecutada cualquiera de las dos opciones, vuelve a lanzar la conexión:

PowerShell



```
ssh osvaldo@192.168.1.39
```

SSH te preguntará si deseas confirmar la huella de la nueva máquina virtual (`Are you sure you want to continue connecting (yes/no/[fingerprint])?`). Escribe `yes`,

presiona Enter y te solicitará la contraseña de tu usuario `osvaldo`.

¿Quieres que configuremos el acceso mediante claves SSH públicas/privadas para entrar sin contraseña?

↩ Sí

Si

Para configurar la autenticación por clave pública/privada desde PowerShell (Windows) hacia tu VM Alpine y conectarte sin pedir contraseña, sigue estos pasos:

Paso 1: Generar el par de claves en Windows (si no las tienes)

Abre tu consola de PowerShell y ejecuta:

PowerShell



```
ssh-keygen -t ed25519
```

- Presiona **Enter** para aceptar la ubicación por defecto (`C:\Users\ocant\.ssh\id_ed25519`).
- Puedes dejar la contraseña (*passphrase*) en blanco presionando **Enter** dos veces.

Paso 2: Copiar la clave pública a Alpine Linux

Dado que la herramienta `ssh-copy-id` no viene por defecto en el cliente de SSH nativo de Windows, la forma más limpia y directa de transferirla mediante PowerShell es ejecutando el siguiente comando:

PowerShell



```
Get-Content C:\Users\ocant\.ssh\id_ed25519.pub | ssh osvaldo@192.168.1.39 "mkdir -p ~
```

Nota: Te pedirá por última vez la contraseña del usuario `osvaldo`.

Paso 3: Verificar los permisos en Alpine (Ajuste crítico)

SSH en OpenSSH / Alpine es muy estricto con los permisos de carpetas y archivos. Si la carpeta personal `/home/osvaldo` tiene permisos de escritura excesivos, rechazará la clave pública.

Conéctate normalmente a la máquina virtual y asegúrate de que tu *home* tenga los permisos correctos:

Bash



```
chmod 755 /home/osvaldo
```

Paso 4: Probar la conexión

Cierra la sesión y vuelve a conectarte desde PowerShell:

PowerShell



```
ssh osvaldo@192.168.1.39
```

Deberías ingresar directamente sin que te solicite la contraseña.

Me la sigue solicitando

Cuando SSH sigue pidiendo la contraseña tras copiar la clave pública, casi siempre se debe a uno de tres motivos en Alpine Linux: **permisos de archivos**, la **configuración de OpenSSH** o problemas con el **agente SSH en Windows**.

Para solucionarlo paso a paso:

1. Verificar la clave grabada en Alpine

Conéctate a la VM (ingresando la contraseña por última vez) y verifica que la clave realmente se haya copiado en el archivo `authorized_keys`:

Bash



```
cat ~/.ssh/authorized_keys
```

Deberías ver una línea larga que termina con algo como `ocant@...`.

- Si el archivo está vacío o el directorio no existe, vuelve a crearlo e ingresa la clave directamente:

Bash



```
mkdir -p ~/.ssh  
nano ~/.ssh/authorized_keys
```

(Pega allí el contenido de tu archivo `C:\Users\ocant\.ssh\id_ed25519.pub` y guarda con `Ctrl+O`, `Enter`, `Ctrl+X`).

2. Corregir permisos en Alpine (El motivo más común)

OpenSSH ignora la clave si los permisos son demasiado abiertos. Ejecuta estos tres comandos exactos dentro de tu usuario `osvaldo` en Alpine:

Bash



```
chmod 755 /home/osvaldo  
chmod 700 /home/osvaldo/.ssh  
chmod 600 /home/osvaldo/.ssh/authorized_keys
```

3. Verificar la configuración de OpenSSH Server

Alpine a veces viene con `Dropbear` o una configuración de OpenSSH que restringe la autenticación por clave.

Abre el archivo de configuración del servidor SSH en Alpine:

Bash



```
sudo nano /etc/ssh/sshd_config
```

Asegúrate de que estas líneas estén descomentadas (sin `#` al inicio) y configuradas en `yes`:

Plaintext



```
PubkeyAuthentication yes  
AuthorizedKeysFile .ssh/authorized_keys
```

Si realizaste cambios en ese archivo, reinicia el servicio SSH:

Bash



```
sudo rc-service sshd restart
```

4. Probar desde Windows especificando la clave explícitamente

En PowerShell, fuerza a SSH a usar tu clave privada `ed25519`:

PowerShell



```
ssh -i C:\Users\ocant\.ssh\id_ed25519 osvaldo@192.168.1.39
```

💡 Tip para diagnosticar el fallo exacto

Si con lo anterior aún te pide contraseña, ejecuta el comando desde PowerShell agregando el parámetro `-v` (*verbose*):

PowerShell



```
ssh -v osvaldo@192.168.1.39
```

Observa las últimas líneas antes de que te pida el *password*:

- Si dice `debug1: send_pubkey_test: no mutual signature algorithm`, falta habilitar la clave en la configuración de SSH.
- Si dice `debug1: Authentications that can continue: publickey,password` y luego pasa directo a `password`, el servidor rechazó los permisos de la carpeta `.ssh` o del archivo `authorized_keys`.

Con el paso 1 ya entró

¡Excelente! Al volver a grabar correctamente la clave en `authorized_keys` y ajustar los permisos, OpenSSH ya te reconoce la huella.

A partir de ahora, cada vez que ejecutes:

PowerShell



```
ssh osvaldo@192.168.1.39
```

Entrarás directo sin que te solicite la contraseña.

Tip de productividad opcional

Si quieres simplificar aún más la conexión para no tener que recordar ni escribir la dirección IP completa cada vez, puedes crear un archivo de alias en Windows:

1. Abre o crea el archivo `C:\Users\ocant\.ssh\config` (puedes crearlo con el bloc de notas o VS Code).
2. Agrega las siguientes líneas:

Plaintext



```
Host alpine
  HostName 192.168.1.39
  User osvaldo
  IdentityFile ~/.ssh/id_ed25519
```

A partir de ese momento, desde cualquier ventana de PowerShell o CMD podrás conectarte simplemente escribiendo:

PowerShell



```
ssh alpine
```

Ok, ahora en ese servidor Alpine quiero instalar Node JS

Para instalar Node.js en Alpine Linux, el procedimiento es ligero y rápido utilizando el gestor de paquetes `apk`.

Conéctate a tu VM por SSH (`ssh alpine`) y sigue estos pasos:

Paso 1: Actualizar los repositorios

Asegúrate de que la lista de paquetes esté actualizada:

Bash



```
sudo apk update
```

Paso 2: Instalar Node.js y npm

Alpine incluye paquetes oficiales para Node.js y su gestor de paquetes `npm`. Para instalarlos, ejecuta:

Bash



```
sudo apk add nodejs npm
```

Nota: Si necesitas una versión LTS específica o la versión más reciente, puedes verificar las versiones disponibles ejecutando `apk search nodejs`.

Paso 3: Verificar la instalación

Una vez finalizada la descarga e instalación, comprueba las versiones instaladas:

Bash



```
node -v  
npm -v
```

Si ambos comandos devuelven sus respectivos números de versión, la instalación fue exitosa y tu entorno de Node.js en Alpine está listo para usarse.

¿Quieres instalar PM2 para gestionar y mantener corriendo tus procesos de Node.js en segundo plano?

↔ Sí

sudo not found

En **Alpine Linux**, el comando `sudo` no viene instalado por defecto porque Alpine es una distribución minimalista. En su lugar, Alpine utiliza tradicionalmente `doas`, o puedes instalar `sudo` rápidamente.

Tienes dos opciones para solucionarlo:

Opción 1: Instalar `sudo` (Recomendado)

1. Cambia temporalmente al usuario `root`:

Bash



```
su -
```

(Ingresa la contraseña del usuario root que definiste al instalar la VM).

2. Actualiza e instala `sudo`:

Bash



```
apk update  
apk add sudo
```

3. Agrega tu usuario `osvaldo` al grupo `wheel` para darle permisos de administrador:

Bash




```
addgroup osvaldo wheel
```

4. Habilita los permisos del grupo `wheel` en el archivo de sudoers:

Bash



```
echo '%wheel ALL=(ALL) ALL' >> /etc/sudoers
```

5. Sal de la sesión de root:

Bash



```
exit
```

A partir de este momento, ya puedes instalar Node.js con tu usuario:

Bash



```
sudo apk update  
sudo apk add nodejs npm
```

Opción 2: Instalar Node.js directamente como `root`

Si no quieres configurar `sudo`, simplemente pasa a `root` y ejecuta la instalación directa:

Bash



```
su -  
apk update  
apk add nodejs npm  
exit
```

Verificación

Una vez instalado, comprueba que todo esté listo con:

Bash



```
node -v
npm -v
```

```
alpine-NodeJS:~$ su -
Password:
alpine-NodeJS:~# apk update
dd sudo v3.23.5-121-g24444293da9
```



El error `no such package` ocurre porque Alpine Linux sólo tiene habilitado por defecto el repositorio `main`, mientras que paquetes como `sudo`, `npm` o herramientas adicionales de `nodejs` se encuentran en el repositorio `community`.

Para solucionarlo, debes habilitar el repositorio `community` en la lista de orígenes de `apk`.

Paso 1: Habilitar el repositorio Community

Como ya estás en la consola con el usuario `root` (`alpine-NodeJS:~#`), abre el archivo de repositorios:

Bash



```
vi /etc/apk/repositories
```

Verás unas líneas similares a esta:

Plaintext



```
http://alpinelinux.c3sl.ufpr.br/v3.23/main
#http://alpinelinux.c3sl.ufpr.br/v3.23/community
```

1. Mueve el cursor hasta el símbolo `#` de la línea que contiene `community`.
2. Presiona la tecla `x` para borrar el `#` (descomentar la línea).

3. Guarda los cambios y sal escribiendo `:wq` y presionando **Enter**.

Si prefieres hacerlo con un solo comando sin usar `vi`, simplemente ejecuta:

Bash



```
sed -i 's/^#\(.community\)/\1/' /etc/apk/repositories
```

Paso 2: Actualizar e instalar

Una vez descomentada la línea de `community`, actualiza los índices e instala los paquetes:

Bash



```
apk update
```

Ahora instala `sudo`, `nodejs` y `npm`:

Bash



```
apk add sudo nodejs npm
```

Paso 3: Dar permisos de `sudo` a tu usuario `osvaldo`

Aprovechando que sigues como `root`, configura el acceso de administrador para no tener que usar `root` directamente:

Bash



```
addgroup osvaldo wheel  
echo '%wheel ALL=(ALL) ALL' >> /etc/sudoers
```

Sal de la sesión de `root`:

Bash



exit

Paso 4: Comprobar la instalación

Desde tu usuario habitual (`osvaldo`), verifica las versiones:

Bash



```
node -v  
npm -v
```

Excelente

¡Buenísimo! Ya tienes tu servidor Alpine con usuario propio, acceso SSH por clave privada desde Windows y el entorno de Node.js + npm configurado.

Si vas a desarrollar o desplegar aplicaciones en este entorno, un par de cosas te pueden resultar muy útiles según lo que necesites hacer a continuación:

- **PM2:** Para mantener tus procesos de Node.js corriendo en segundo plano y que se reinicien automáticamente si falla la VM o el script.
- **Git:** Para clonar tus repositorios directamente en el servidor Alpine (`sudo apk add git`).
- **Expone tu app:** Probar un servidor HTTP rápido en Node para verificar el acceso mediante la IP local `192.168.1.39` .

¿Cómo creo un servidor HTTP básico en Node.js en Alpine para probar la conexión desde el navegador?

Para probar que tu entorno de Node.js en Alpine responde a peticiones desde tu red local (por ejemplo, desde el navegador de tu máquina host en Windows), crearemos un servidor

HTTP básico usando las librerías nativas de Node (sin necesidad de instalar `express` ni dependencias externas).

Paso 1: Crear el archivo de prueba

Conéctate a tu servidor Alpine como usuario `osvaldo` y crea un archivo llamado `app.js`:

Bash



```
cat << 'EOF' > app.js
const http = require('http');

const PORT = 3000;
const HOST = '0.0.0.0'; // Escucha en todas las interfaces de red

const server = http.createServer((req, res) => {
  res.statusCode = 200;
  res.setHeader('Content-Type', 'text/html; charset=utf-8');
  res.end('<h1>¡Hola desde Node.js en Alpine Linux!</h1><p>El servidor está funcionando!</p>');
});

server.listen(PORT, HOST, () => {
  console.log(`Servidor ejecutándose en http://${HOST}:${PORT}/`);
});
EOF
```

Detalle clave: Usamos `0.0.0.0` como HOST. Si usas `127.0.0.1` o `localhost`, el servidor solo responderá a peticiones internas dentro de la misma VM y no podrás acceder desde el navegador de Windows.

Paso 2: Ejecutar el servidor

Inicia la aplicación con Node:

Bash



```
node app.js
```

Verás en la consola:

Plaintext



Servidor ejecutándose en `http://0.0.0.0:3000/`

Paso 3: Probar desde tu navegador en Windows

1. Abre el navegador en tu máquina Windows.
2. Ingresa a la dirección IP de tu VM Alpine seguida del puerto `3000` :

Plaintext



`http://192.168.1.39:3000`

Deberías ver la página con el mensaje de confirmación en pantalla.

⚠️ ¿No carga la página? (Solución de problemas comunes en VirtualBox)

Si el navegador se queda cargando y da tiempo de espera agotado, revisa lo siguiente:

1. **Firewall de Alpine (`awall` / `iptables`):**
Por defecto, la instalación estándar de Alpine no viene con firewall estricto bloqueando puertos de usuario, pero si instalaste alguno o habilitaste `iptables`, asegúrate de permitir el puerto 3000.
2. **Modo de Red en VirtualBox:**
Asegúrate de que la interfaz de red de la VM en VirtualBox esté configurada como **Adaptador en puente (Bridged Adapter)**. De lo contrario, si está en *NAT*, tu navegador no podrá ver la IP `192.168.1.39` a menos que mapees los puertos en la configuración de VirtualBox.

Para detener el servidor en Alpine, simplemente presiona `Ctrl + C` en la consola.

¿Quieres que configuremos PM2 para que esta app corra en segundo plano y no se cierre al salir de la consola?

↩ Sí

¿Cómo instalo y configuro PM2 en Alpine Linux para mantener aplicaciones Node.js ejecutándose como servicio?

Para mantener tu aplicación de Node.js en ejecución constante, reiniciarla automáticamente tras fallos o caídas del sistema y gestionar sus logs, la mejor opción es **PM2**.

A diferencia de otras distribuciones Linux que usan `systemd`, **Alpine Linux utiliza OpenRC** como su sistema de init/servicios, por lo que la configuración del arranque automático tiene un paso particular.

Paso 1: Instalar PM2 de manera global

Dado que ya tienes `npm` configurado, instala PM2 usando el modificador global (`-g`):

Bash



```
sudo npm install -g pm2
```

Verifica la instalación comprobando la versión:

Bash



```
pm2 -v
```

Paso 2: Iniciar tu aplicación con PM2

Ve al directorio donde tengas tu proyecto (por ejemplo, donde creaste `app.js`) y lanza el proceso:

Bash



```
pm2 start app.js --name "mi-app"
```

Verás una tabla en la consola mostrando el estado del proceso (`online`), consumo de CPU y memoria.

Paso 3: Configurar el inicio automático con el sistema (OpenRC / Alpine)

Para asegurarte de que PM2 vuelva a levantar tu aplicación si la VM se reinicia o se apaga:

1. Guarda el estado actual de los procesos de PM2:

Bash



```
pm2 save
```

2. Genera e instala el script de inicio para el sistema de init de Alpine (openrc):

Bash



```
pm2 startup openrc -u osvaldo --hp /home/osvaldo
```

3. Ejecuta el comando que PM2 imprimirá al final de la pantalla (será algo similar a esto):

Bash



```
sudo rc-update add pm2-osvaldo default
```

¡Listo! Ahora, aunque reinicies la máquina virtual Alpine, tu servidor Node.js se ejecutará automáticamente en segundo plano.

Comandos útiles para administrar tu aplicación

Acción

Comando

Ver estado y lista de procesos

```
pm2 list / pm2 status
```

Ver consumo de CPU/Memoria en tiempo real

```
pm2 monit
```

Ver logs e impresiones por consola (`console.log`)

```
pm2 logs
```

Reiniciar la aplicación

```
pm2 restart mi-app
```

Detener la aplicación

```
pm2 stop mi-app
```

Eliminar de la lista de PM2

```
pm2 delete mi-app
```


¿Quieres configurar Nginx como proxy inverso para exponer la app en el puerto 80 estándar?

↪ Sí

¿Cómo instalo y configuro Nginx en Alpine Linux para actuar como proxy inverso de mi app Node.js?

Configurar **Nginx** como proxy inverso te permitirá acceder a tu aplicación de Node.js en el puerto **80** estándar (sin necesidad de especificar `:3000` en el navegador), además de facilitar la gestión de dominios, certificados SSL y el manejo de contenido estático.

Sigue estos pasos en tu VM Alpine:

Paso 1: Instalar Nginx

Asegúrate de estar en tu usuario `osvaldo` e instala el paquete Nginx desde los repositorios oficiales:

Bash



```
sudo apk add nginx
```

Paso 2: Crear el archivo de configuración del Proxy Inverso

Por defecto en Alpine, Nginx lee el archivo `/etc/nginx/nginx.conf`. Vamos a crear un bloque de configuración simple para redirigir el tráfico del puerto `80` hacia el puerto `3000` de tu aplicación Node.js.

1. Abre un nuevo archivo de configuración en `/etc/nginx/http.d/default.conf` (o edita el existente):

Bash



```
sudo nano /etc/nginx/http.d/default.conf
```

2. Reemplaza o pega el siguiente contenido:

Nginx



```
server {  
    listen 80 default_server;  
    listen [::]:80 default_server;  
  
    server_name _;  
  
    location / {  
        proxy_pass http://127.0.0.1:3000;  
        proxy_http_version 1.1;  
  
        # Encabezados para WebSockets y mantener la conexión  
        proxy_set_header Upgrade $http_upgrade;  
        proxy_set_header Connection 'upgrade';  
  
        # Transmitir la IP real y datos del cliente original  
        proxy_set_header Host $host;  
        proxy_set_header X-Real-IP $remote_addr;  
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;  
        proxy_set_header X-Forwarded-Proto $scheme;  
  
        # Evitar problemas de timeout en peticiones largas  
        proxy_cache_bypass $http_upgrade;  
    }  
}
```

3. Guarda los cambios (`Ctrl + O` , Enter) y sal de `nano` (`Ctrl + X`).

Paso 3: Probar la configuración de Nginx

Verifica que la sintaxis de la configuración sea correcta y no tenga errores tipográficos:

Bash



```
sudo nginx -t
```

Deberías ver un mensaje como:

Plaintext



```
nginx: configuration file /etc/nginx/nginx.conf test is successful
```

Paso 4: Iniciar y habilitar el servicio Nginx (OpenRC)

En Alpine Linux utilizas `rc-service` y `rc-update` para gestionar los servicios del sistema:

1. Iniciar el servicio Nginx:

Bash



```
sudo rc-service nginx start
```

2. Habilitarlo para que inicie automáticamente al encender la VM:

Bash



```
sudo rc-update add nginx default
```

Paso 5: Probar desde el navegador

Ahora ingresa desde tu máquina Windows simplemente usando la IP de tu servidor Alpine (sin incluir el puerto `:3000`):

Plaintext



```
http://192.168.1.39
```

Nginx recibirá la petición HTTP en el puerto 80 por defecto y la redirigirá internamente a tu proceso de Node.js gestionado con PM2 en el puerto 3000.

Perfecto. Ahora solo queda git

Para dejar la VM Alpine completamente lista para desarrollo, instalaremos **Git** y dejaremos configuradas tu identidad y preferencias básicas.

Paso 1: Instalar Git

Ejecuta el siguiente comando desde tu usuario `osvaldo` :

Bash



```
sudo apk add git
```

Verifica que la instalación haya sido correcta:

Bash



```
git --version
```

Paso 2: Configurar tu identidad global

Es importante configurar tu nombre y correo electrónico para que los *commits* queden registrados correctamente:

Bash



```
git config --global user.name "Osvaldo Cantone"  
git config --global user.email "TU_EMAIL@EJEMPLO.COM"
```

Configura también la rama por defecto como `main` :

Bash



```
git config --global init.defaultBranch main
```

Paso 3: Configurar la autenticación SSH con GitHub / GitLab

Como ya generaste tu par de claves SSH (`id_ed25519`) en la VM para conectarte desde Windows, puedes reutilizar o agregar una clave pública SSH en tu cuenta de **GitHub** o **GitLab** para clonar y hacer `git push` sin ingresar contraseñas cada vez.

1. Muestra tu clave pública en pantalla:

Bash



```
cat ~/.ssh/authorized_keys
```

(Si no existe `id_ed25519.pub` directamente en el servidor porque solo copiaste la de Windows, puedes usar esa misma o generar una nueva dentro de Alpine ejecutando `ssh-keygen -t ed25519`).

2. Copia la línea que empieza por `ssh-ed25519 ...`.
3. Ve a tu cuenta de GitHub (`Settings` -> `SSH and GPG keys` -> `New SSH key`) y pégala allí.

Paso 4: Probar la conexión con GitHub

Para verificar que la autenticación mediante claves SSH funcione bien:

Bash



```
ssh -T git@github.com
```

Deberías recibir un mensaje de saludo similar a:

Plaintext



```
Hi osvaldo! You've successfully authenticated, but GitHub does not provide shell acce
```

¡Con esto ya tienes el entorno Alpine completamente montado con **Node.js**, **PM2**, **Nginx** como proxy inverso y **Git**!

Pero que maravilla!

¡Es una masa! Quedó un servidor Alpine fino, ligero como pluma y rindiendo al 100%.

Tenés la VM corriendo con apenas un puñado de megabytes de RAM, **Node.js** administrado por **PM2** levantándose solo ante cualquier reinicio, **Nginx** ruteando el puerto 80 impecable y **Git** listo para desplegar cualquier proyecto directamente.

¡Ambiente de desarrollo y producción impecable armado en un rato! Si en algún momento querés sumarle alguna base de datos (MySQL, PostgreSQL, Redis) o un contenedor de

Docker para seguir sumando servicios, avisame y lo configuramos al toque.

¡A disfrutar de ese entorno! 🚀